



GO

Overview of Go Language

Less is More

GO

Function



- Functions are declared with `func` keyword
- Functions can take zero or more arguments and can return values

```
func main (){  
    fmt.Println("Hello from Go")  
}
```

```
func displayMessage(message string){  
    fmt.Println(message)  
}
```

```
displayMessage("Hello")
```

```
func add(x int, y int) int {  
    return x+y  
}
```

```
var result int = add(20,10)  
fmt.Println(result)
```

Functions (cont'd)



- When function arguments are of same type it could be shortened -

```
// func add(x int, y int) int {
```

```
func add(x, y int) int {  
    return x+y  
}
```

```
var result int = add(20,10)  
fmt.Println(result)
```

Anonymous Functions



- Go supports Anonymous Functions

Normal Function

```
package main
import "fmt"

func main(){
    var res int = add(20,10)
    fmt.Println(res)    // 30
}

// add is declared as normal (non
// anonymous function)
func add(x int, y int) int {
    return x+y
}
```

Anonymous Function

```
package main
import "fmt"

// add is declared as non anonymous function
add := func(x int, y int) int {
    return x+y
}

fmt.Println(add(20,10))    // 30

var sub = func(x int, y int) int {
    return x-y
}

fmt.Println(sub(20,10))    // 10
```

Anonymous Functions (cont'd)



- Anonymous Functions could be called immediately
- Very similar to JavaScript (and other functional languages)

```
func main(){  
    func(msg string){  
        fmt.Println("Hello " + msg)  
    }("Aniruddha")  
}  
// Displays Hello Aniruddha
```

```
func main(){  
    func(num1 int, num2 int){  
        fmt.Println(num1 + num2)  
    }(10, 20)  
}  
// Displays 30
```

In Brown – Anonymous Function Declaration

In Green - Anonymous Function Invocation

Higher order functions



- Anonymous Functions could be passed as argument to other functions, and could be returned from other functions. Functions behave like values – could be called function values

```
func main () {  
    sayHello := func() {  
        fmt.Println("Hello from Go")  
    }  
    doWork(sayHello)  
}  
  
func doWork(anonymous func() ){  
    anonymous()  
}  
  
// Displays "Hello from Go"
```

Higher order functions (cont'd)



```
func main () {  
    add := func(x int, y int) int {  
        return x+y  
    }  
  
    sub := func(x int, y int) int {  
        return x-y  
    }  
  
    result := doWork(add, 30, 20)  
    fmt.Println(result)                                // 1300  
  
    result = doWork(sub, 30, 20)  
    fmt.Println(result)                                // 500  
}  
  
func doWork(anonymous func(int, int) int, num1 int, num2 int ) int{  
    return anonymous(num1 * num1, num2 * num2)  
}
```


Higher order functions - User-Defined Function types



```
func main () {  
    add := func(x int, y int) int {  
        return x+y  
    }  
  
    sub := func(x int, y int) int {  
        return x-y  
    }  
  
    result := doWork(add, 30, 20)  
    fmt.Println(result)                                // 1300  
  
    result = doWork(sub, 30, 20)  
    fmt.Println(result)                                // 500  
}  
  
type HigherFunc func(x int, y int) int                // User defined function type  
  
func doWork(anonymous HigherFunc, num1 int, num2 int ) int{  
    return anonymous(num1 * num1, num2 * num2)  
}
```

Variadic Functions



- Variadic functions can be called with any number of trailing arguments.

```
func displayMessage(message string, times int, params ...string){  
    fmt.Println(message, times, params)  
}
```

```
displayMessage("Call1", 1)  
displayMessage("Call2", 2, "Param1")  
displayMessage("Call3", 3, "Param1", "Param2", "Param3", "Param4")
```

Output:

```
Call1 1 []  
Call2 2 [Param1]  
Call3 3 [Param1 Param2 Param3 Param4]
```

GO

Control Structure - If, For and Switch



- If
- For
- Switch

Go does not supports while or do while keywords, though loops similar to while could be written using for

If, else if, else



- The if statement looks as it does in C or Java, except that the () are gone and the { } are required.

```
var salary int = 100
```

```
if salary < 50 {  
    fmt.Println("you are underpaid")  
} else if salary >= 50 {  
    fmt.Println("you are sufficiently paid")  
} else {  
    fmt.Println("you are overpaid")  
}
```



- Go has only one looping construct, the for loop.
 - Go does not have while, do while or for each / for in loops
- The basic for loop looks as it does in C or Java, except that the `( )` are gone (they are not even optional) and the `{ }` are required.

```
for ctr := 0; ctr < 10; ctr++ {  
    fmt.Println(ctr)  
}
```

- Go does not support while or do while. Same could be achieved using for

```
var ctr int = 0  
  
// same as while  
for(ctr < 5) {  
    fmt.Println(ctr)  
    ctr++  
}
```



- As in C or Java, you can leave the pre and post statements empty

```
ctr:=0
for ; ctr < 10; {
    ctr +=1
    fmt.Println(ctr)
}
```

- Semicolons could be dropped: C's while is spelled for in Go

```
ctr:=0
for ctr < 10 {          // behaves in the same way as while ctr < 100 in C or Java
    ctr +=1
    fmt.Println(ctr)
}
```

- Endless or forever loop

```
for {
    // do something - this loop would never end
}
```



- Go's switch statement is more general than C's - expressions need not be constants or even integers.

```
city := "Kolkata"
```

```
switch city {  
case "Kolkata":  
    println("Welcome to Kolkata")  
    break  
case "Bangalore":  
    println("Welcome to Bangalore")  
    break  
case "Mumbai":  
    println("Welcome Mumbai")  
    break  
}
```

```
rating := 2
```

```
switch rating {  
case 4:  
    println("You are rated Excellent")  
    break  
case 3:  
    println("You are rated Good")  
    break  
case 2:  
    println("You are rated Consistent")  
    break  
case 1:  
    println("You need to improve a bit")  
    break  
}
```


Type switch



- Used to discover the dynamic type of an interface variable. Such a type switch uses the syntax of a type assertion with the keyword `type` inside the parentheses.
- If the switch declares a variable in the expression, the variable will have the corresponding type in each clause. It's also idiomatic to reuse the name in such cases, in effect declaring a new variable with the same name but a different type in each case.

```
type Human interface {
    Display()
}

type Employee struct {
    name string; designation string
}

func (emp Employee) Display(){
    fmt.Println(emp.name, emp.designation)
}

type Contractor struct {
    name string; weeklyHours int
}

func (cont Contractor) Display(){
    fmt.Println(cont.name, cont.weeklyHours)
}
```

```
func main() {
    var human Human
    human = Employee
    {name:"Aniruddha",designation:"AVP"}

    switch human := human.(type){
        default:
            fmt.Println("default")
        case Employee:
            fmt.Println("Human",
                human.designation)
        case Contractor:
            fmt.Println("Cont", human.weeklyHours)
    }
}
```